

Project Aim

The core objective of the **ble_time_sync** project is to design and deploy a highly accurate, event-driven, cross-protocol time synchronization network. The system bridges a Wi-Fi-based MQTT environment with an ultra-low-energy Bluetooth (BLE) topology, creating a fully autonomous, self-healing network dashboard. It features a maximum data latency of under 2 seconds and guarantees absolute chronological accuracy across multiple hardware platforms without any manual intervention.

Equipment Profile & System Architecture

1. The Edge Node & MQTT Publisher

- **Hardware:** Adafruit ESP32S3-TFT (Product ID: 5483).
- **Role:** Master Network Time Acquisition & Sensor Hub.
- **Functional Profile:**
 - Connects via Wi-Fi to fetch absolute internet network time (NTP Epoch timestamp tokens).
 - Continuously interrogates a Pimoroni Multi Sensor Stick (PIM745) to gather environment metrics.
 - Synchronizes and maintains local hardware state clocks via an external target controller (M5Stack Unit RTC 8563).
 - Packages the structural information into a compact MQTT JSON payload stream and publishes it over the local network exactly once every 60 seconds.

2. The Smart Data Gateway (Raspberry Pi CM5)

- **Hardware:** Raspberry Pi Compute Module 5 (Pi CM5).
- **Role:** Multi-Protocol Interpreter & Headless Process Automated Dispatcher.
- **Functional Profile:**

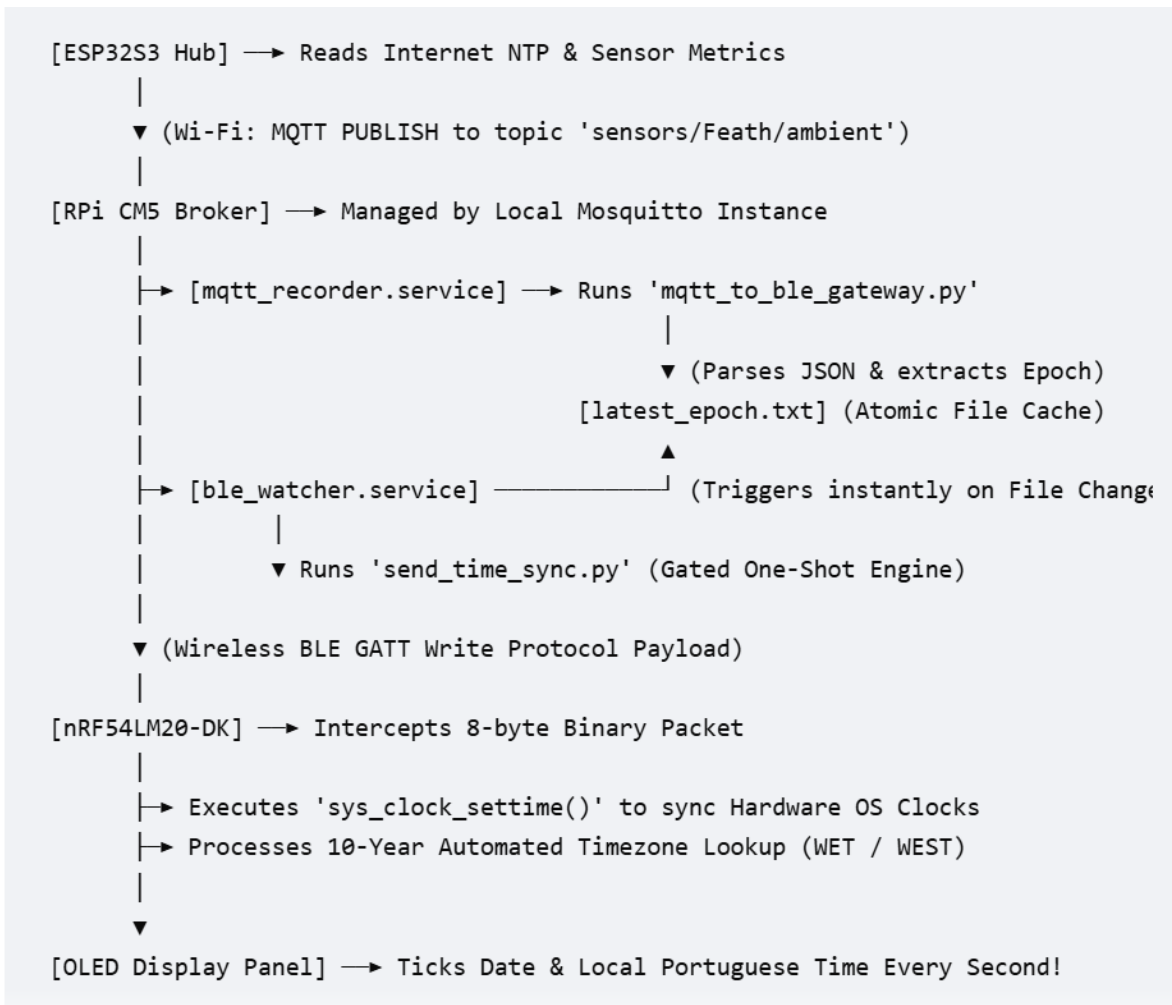
- Runs a central local Mosquitto MQTT Broker instance to intercept incoming Wi-Fi payloads.
- Governs system automated loops using two decoupled background **systemd** process container services:
 - **mqtt_recorder.service**: Executes the background daemon `mqtt_to_ble_gateway.py`. This script streams data continuously from the broker, extracts the raw nested NTP epoch value from the JSON payload, and writes the numeric data to a flat cache text file (`latest_epoch.txt`).
 - **ble_watcher.service**: Runs an unbuffered loop (`send_time_sync.py`) that monitors the file-system modification markers of `latest_epoch.txt` on a strict 1-second cadence. The exact millisecond the timestamp shifts, it triggers an event, reads the integer value, opens a secure BLE connection to the final receiver, and exits cleanly. It includes a built-in 110-second safety filter to prevent any radio channel overlap errors.

3. The End Node & High-Precision Display (nRF54 Target)

- **Hardware:** Nordic Semiconductor nRF54LM20-DK (nRF54LM20B Development Board).
- **Role:** Low-Power Receiver, Hardware Clock Controller, and Display Dashboard.
- **Functional Profile:**
 - Executes a dedicated, optimized firmware image named **ble_time_sync** built inside the modern nRF Connect SDK v3.4.0 (Zephyr OS v4.4.0) ecosystem.
 - Exposes custom 128-bit Vendor-Specific BLE GATT profiles to wait for incoming time synchronization data packets.
 - Intercepts the raw 8-byte little-endian binary stream (`uint64_t`) and passes it into Zephyr's internal system clock manager via **`sys_clock_settime()`**, securely updating the board's internal real-time hardware reference layers.
 - Runs a 10-year automated lookup matrix mapping **Europe/Lisbon** timezone boundaries to dynamically toggle between Western European Time (**WET**) and Western European Summer Time (**WEST**).

- Drives a 1.12" Adafruit SH1107 OLED glass surface panel over I2C, rendering a clean visual layout showing the active status header, lines, a dynamic calendar date row (yyyy-mm-dd), the calculated time prefix, and a real-time ticking clock display (HH:MM:SS) updating smoothly every second.

Data Interconnection Diagram



Critical System Advantages

- **Decoupled Architecture:** Separating the MQTT network tasks from the BLE radio transactions completely eliminates Device Busy (-11) collisions on the Pi's BlueZ stack.
- **Self-Healing Automation:** Both Pi tasks are registered as system services (`mqtt_recorder.service` and `ble_watcher.service`) that automatically spin up at boot time and recover instantly if a power failure occurs.
- **Hardware-Level Integration:** Utilizing native Zephyr kernel clocks ensures that the nRF54 target maintains chronologically perfect internal system timing references down to the microsecond.

System Hardware Summary

The **ble_time_sync** project relies on three distinct computing tiers, bridging a high-frequency sensor layer with a low-power wireless display node over an intelligent Linux gateway.

Table 1

(on next page)

Hardware Platform	Primary Architecture & Key Specs	Project Role & Implementation
Adafruit Feather ESP32-S3 TFT (<i>Product ID: 5483</i>)	Tensilica Xtensa LX7 dual-core processor, integrated 2.4 GHz Wi-Fi/BLE, built-in color 240x135 IPS TFT display.	MQTT Publisher (Wi-Fi Core): Connects to network routers to fetch global internet NTP Epoch metrics and broadcasts data strings once a minute
Pimoroni Multi Sensor Stick (<i>Product ID: PIM745</i>)	Bosch BME280 temperature, pressure, humidity sensor I2C interface (address: 0x76)	Environmental Sensor Hub: Monitors real-time ambient temperature, pressure and relative humidity properties.
M5Stack Unit RTC (<i>Product ID: U126</i>)	NXP PCF8563 high-stability Real-Time Clock calendar chip with backed backup battery infrastructure.	Local Hardware Clock Tracker: Maintained by the ESP32-S3 as a highly stable, non-volatile time anchor framework.
Pimoroni Qwiic I2C Gamepad Controller (<i>PIM752</i>)	• TCA9555 I/O Expander • 8x large user buttons (labelled U, D, L, R, A, B, X and Y) • 2x small user buttons (labelled + and -) • 4x white LEDs • 2x Qw/ST (Qwiic/STEMMA QT) connectors • I2C interface, with addresses: 0x21 (default), 0x23, 0x25 or 0x27 • 3V to 5V compatible	To command the MQTT Publisher app.
Raspberry Pi Compute Module 5 (<i>Pi CM5</i>)	Broadcom BCM2712 quad-core ARM Cortex-A76 CPU @ 2.4GHz, modern architecture, dual-channel BLE 5.0 wireless controller.	System Gateway & Broker: Runs a Linux OS instance executing persistent systemd cross-protocol pipelines: mqtt_recorder.service, ble_watcher.service and a MQTT broker (Mosquitto).

Nordic nRF54LM20-DK (<i>nRF54LM20B</i>)	ARM Cortex-M33 application processor up to 128 MHz, sub- μ A low-power tracking registers, advanced security engine.	End-Node Receiver: Runs the Zephyr RTOS ble_time_sync image, maps 10-year timezone matrices, and updates internal clocks via hardware abstraction
Adafruit 1.12" OLED Display (Product ID: 5297)	Solomon Systech SH1107 driver controller, high-contrast monochrome 128x128 white pixels over I2C lines.	Dynamic Glass Interface Dashboard: Displays the active link status headers, calendar parameters, and a high-precision ticking clock.

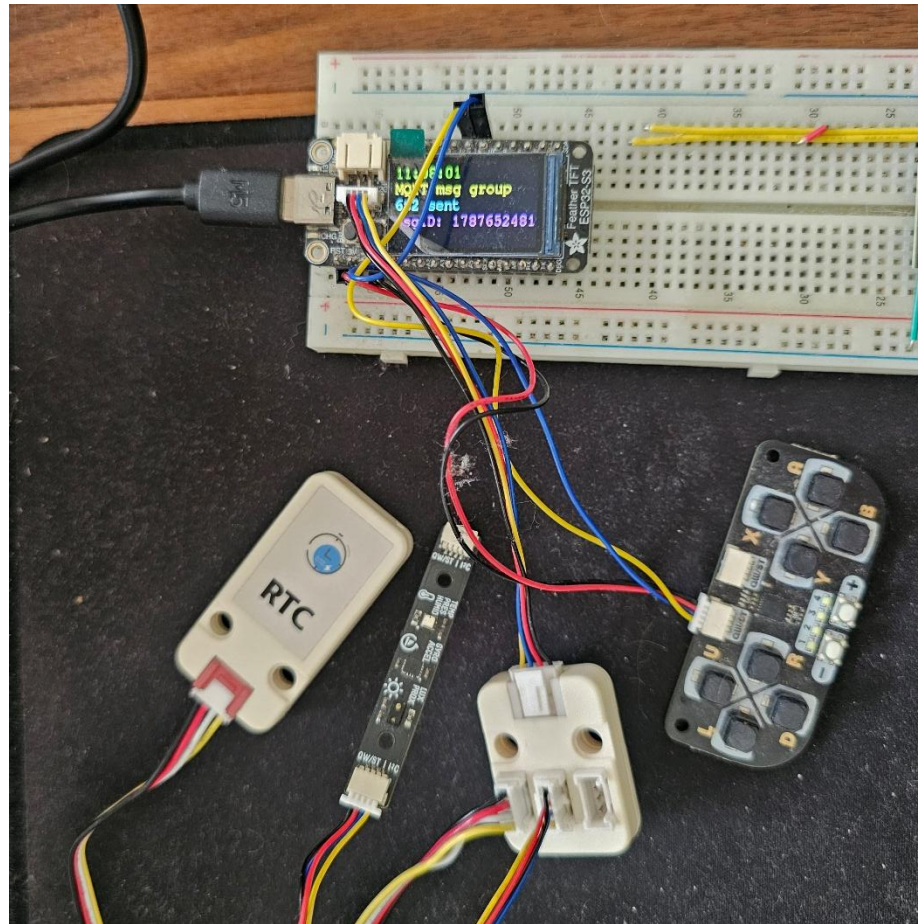
Key Hardware Interoperability Features

- **Cross-Protocol Data Translation:** Translates 2.4 GHz Wi-Fi data frames (MQTT protocol) down to local Linux file cache markers, then packages them into low-energy Bluetooth (BLE) GATT wireless packets.
- **Decoupled Architecture:** Using physical text file caching separates the network-heavy MQTT transactions from the BLE radio controller, completely eliminating hardware collisions.
- **Low-Power Precision:** The nRF54 core uses an internal hardware clock structure, allowing it to drive the pixel display and calculate timezone shifts with exceptional power efficiency.

Images

1. Adafruit ESP32-S3-TFT (prod.ID 5483) with Pimoroni multi-sensor-stick (PIM745); M5Stack RTC Unit and Pimoroni Qwiic I2C Gamepad Controller (PIM752); M5Stack 3x Grove Hub.

Function: MQTT Publisher





2. Raspberry Pi
Compute Module 5
with SSD.

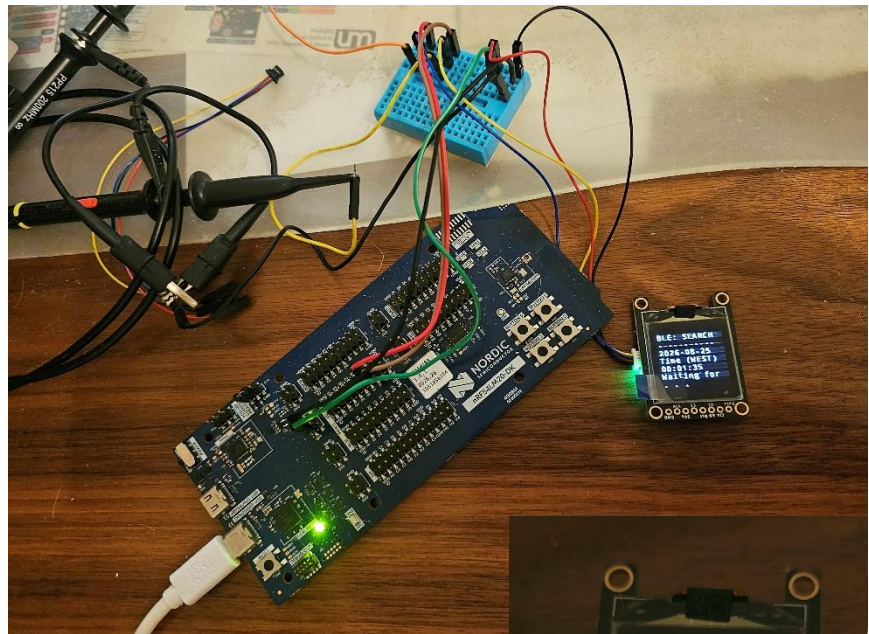
Functions:

- MQTT Broker;
- MQTT to BLE gateway;
- BLE Watcher
(send_time_sync)

3. Nordic nRF54LM20-
DK (nRF54LM20B) and
1.12" 128x128 mono
OLED.

Functions:

- BLE msg interpreter;
- Set internal RTC;
- Display text on OLED



Project: ble_time_sync

by Paulus Schulinck (Github @PaulskPt) 2026-08-24

Lisbon, 2026-08-24

Paulus Schulinck (Github @PaulskPt)

Recognition:

I would not have been able to realize this project without the great help of Google's AI assistant Gemini. The assistance of Gemini made it possible to create this project in three days over a weekend.

Visit my Github [repositories](#)